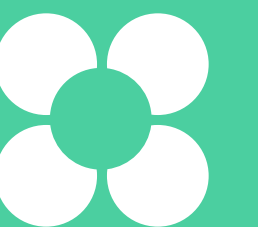


# Отправка и получение данных через канал

Матвей Ефимов

Разработчик учебных программ и эксперт в Go, Python, SQL



# Матвей Ефимов

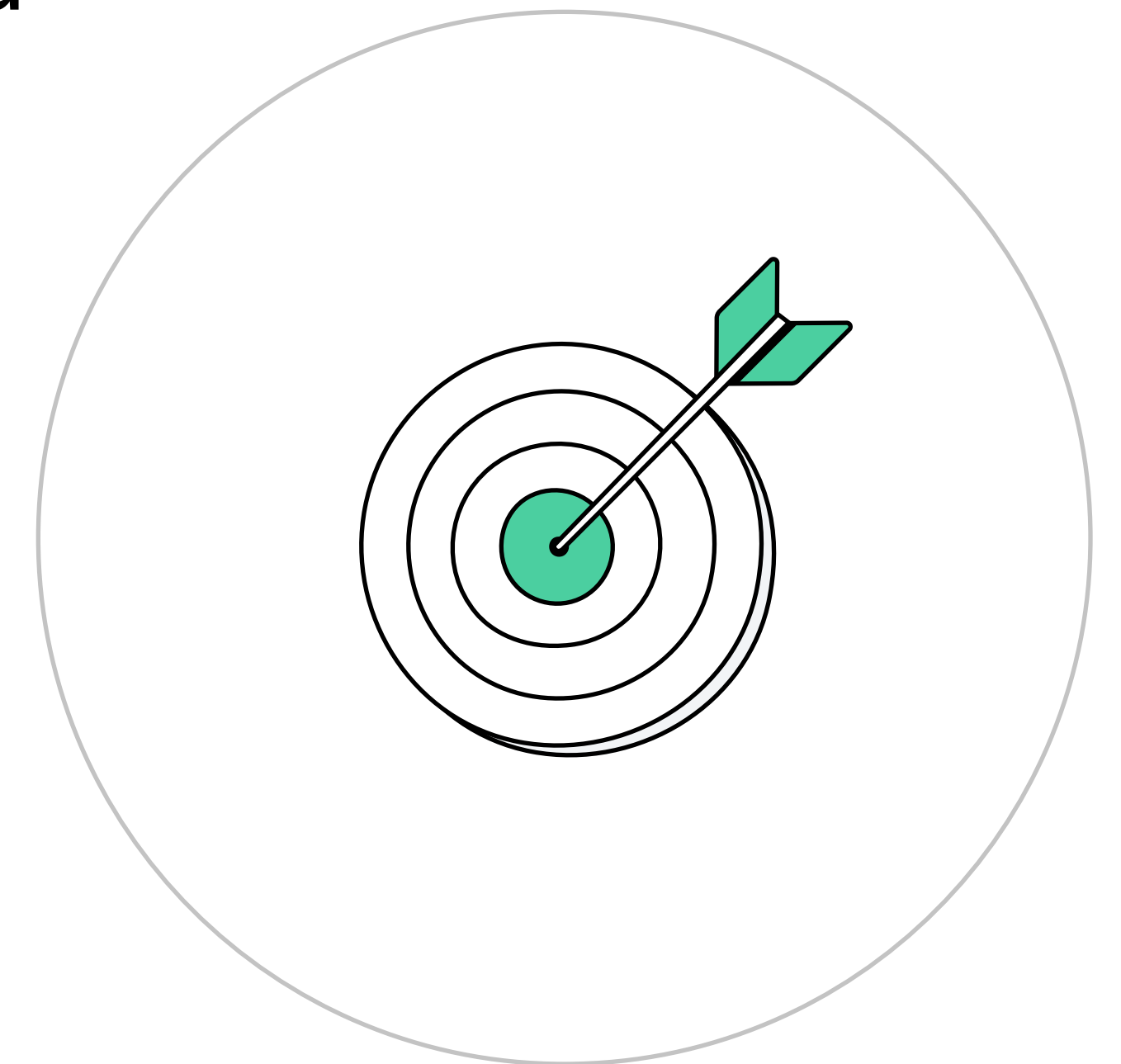
О спикере:

- преподаватель с более чем 5-летним опытом в IT-образовании
- разработчик учебных программ и эксперт в Go, Python, SQL
- фриланс-программист и наставник студентов от школ до вузов



# Цели занятия

- Разберёмся, как работают каналы в Go и для чего они используются
- Узнаем, как создавать каналы и передавать данные между горутинами
- Поймём, как безопасно читать данные из канала с помощью конструкций `range` и `val, ok := <-ch`
- Разберёмся, как закрывать канал и контролировать завершение работы горутинов
- Узнаем, какие ошибки чаще всего возникают при работе с каналами и как их избегать
- Поймём, как устроены базовые паттерны взаимодействия горутинов через каналы



# План занятия

- 1 Взаимодействие горутин через каналы
- 2 Работа с данными в канале:  
структуры и завершение
- 3 Работа с каналами: код, поведение  
горутин и типичные ошибки
- 4 Итоги

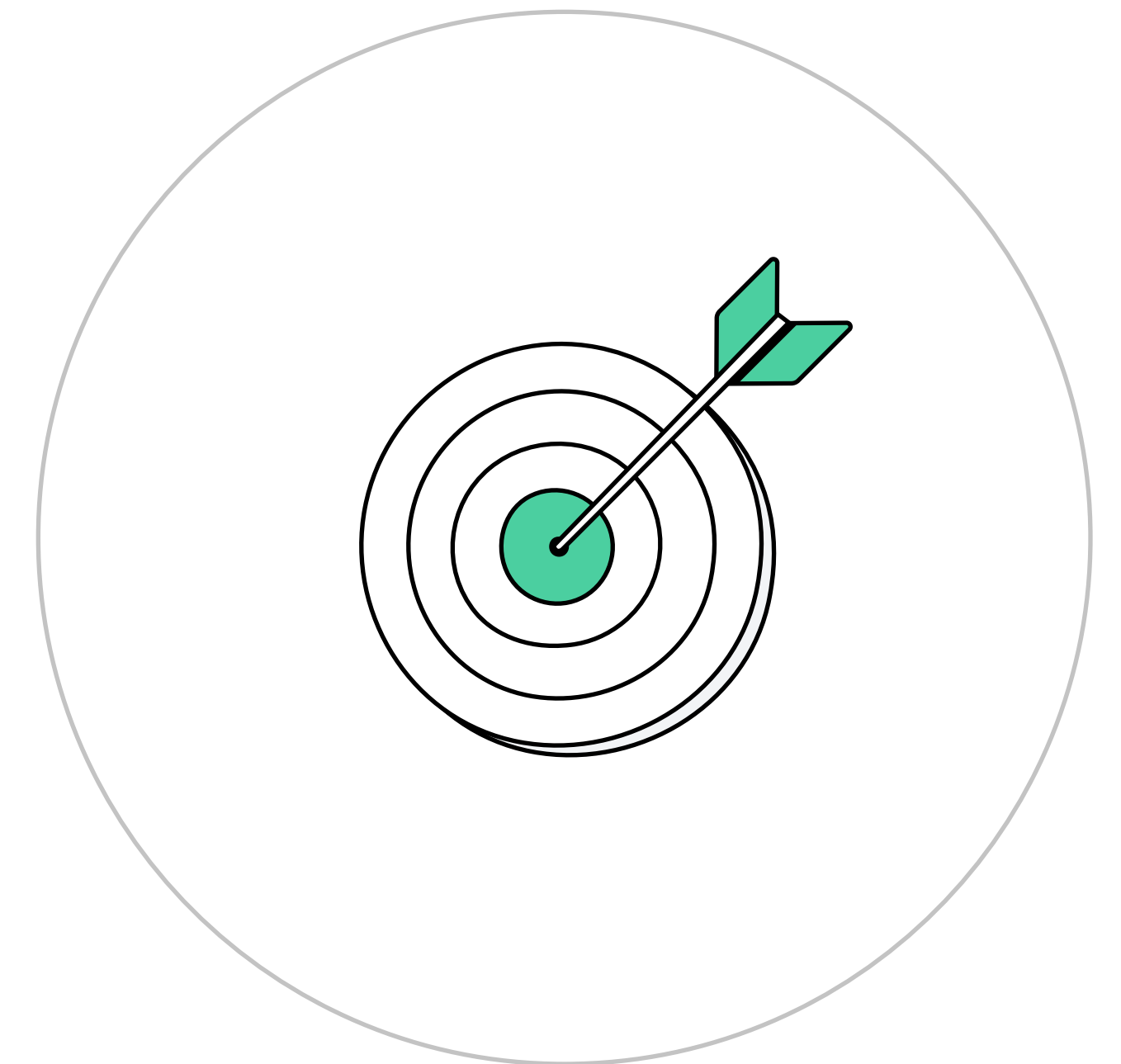


# Взаимодействие горутин через каналы



# Цели видео

- Разберёмся, как реализовать взаимодействие между горутинами с помощью каналов
- Сосредоточимся на коде и практической работе с каналами
- Поймём, как устроено поведение программы при передаче данных через канал
- Разберём типичные практические приёмы работы с каналами



# Как выглядит базовый синтаксис каналов

Базовый шаблон создания и работы с каналом

```
ch := make(chan int) // создаём канал для int
ch <- 42              // отправляем 42 в канал
x := <-ch             // читаем значение из канала и сохраняем в x
```

# Что важно помнить о каналах

- Каналы позволяют безопасно обмениваться данными между горутинами
- Важно не просто использовать каналы, а использовать их правильно и эффективно

# Пример

- Одна горутина отправляет данные
- Вторая горутина читает данные из канала

## Пример кода передачи данных

```
package main

import (
    "fmt"
)

func main() {
    ch := make(chan string)

    go func() {
        ch <- "Привет от горутины!"
    }()

    msg := <-ch

    fmt.Println("Получено:", msg)
}
```

# Что происходит в коде

## Что делает программа

- Создаём канал `ch` для строк
- Запускаем горутину, которая отправляет строку
- В функции `main` читаем из канала и выводим сообщение

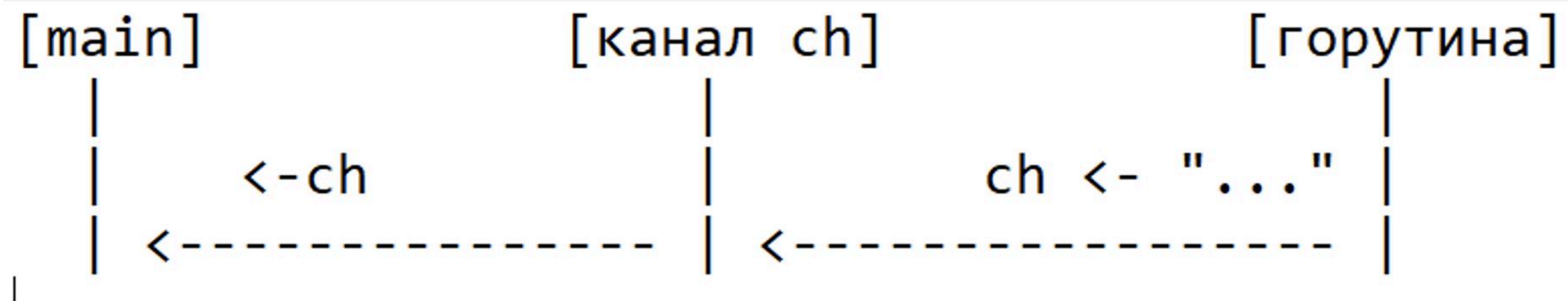
## На что важно обратить внимание

- Если читать из канала до отправки значения — программа заблокируется
- Канал — это синхронная очередь

# Как выглядит канал в памяти

Канал можно представить как трубу

- С одной стороны горутина говорит:  
«Вот тебе значение»
- С другой стороны горутина либо ждёт,  
либо сразу читает данные



# Пример «ping-pong»: две горутины, два канала

```
package main
import (
    "fmt"
    "time"
)
func main() {
    ping :=make(chan string)
    pong :=make(chan string)
    go func() {
        for {
            msg :=<-ping
            fmt.Println("Ping получил:", msg)
            time.Sleep(500 * time.Millisecond)
            pong <- "от ping"
        }
    }()
}
```

```
go func() {
    for {
        msg := <-pong
        fmt.Println("Pong получил:", msg)
        time.Sleep(500 * time.Millisecond)
        ping <- "от pong"
    }
}()
// запускаем пинг
ping <- "старт"
// ждём немного, чтобы увидеть, как идёт обмен
time.Sleep(3 * time.Second)
fmt.Println("Завершение")
}
```

# Пример «ping-pong»

- Используются два канала: `ping` и `pong`
- Первая горутина ждёт сообщение в `ping`, затем отправляет ответ в `pong`
- Вторая горутина ждёт сообщение в `pong`, затем отправляет ответ в `ping`
- В `main()` один раз отправляется стартовое сообщение
- Дальше цикл пинг-понга поддерживается между двумя горутинами
- Каналы помогают не только передавать данные, но и координировать порядок действий между горутинами

# Направленные каналы: chan<- и <-chan

- ping chan<- string — канал только на отправку
- ping <-chan string — канал только на чтение
- Попытка прочесть из chan<- или записать в <-chan вызовет ошибку компиляции
- Направленные каналы помогают избежать логических ошибок, особенно в больших проектах

```
func send(ping chan <- string, msg string) {  
    ping <- msg  
}  
  
func receive(ping <-chan string) {  
    fmt.Println("Получено:", <-ping)  
}  
  
func main() {  
    ch := make(chan string)  
    go send(ch, "сообщение")  
    go receive(ch)  
    time.Sleep(time.Second)  
}
```

# Пример с пулом воркеров

```
package main

import (
    "fmt"
    "time"
)

func worker(id int, jobs <-chan int) {
    for job := range jobs {
        fmt.Printf("Worker %d начал задание %d\n", id, job)
        time.Sleep(time.Second)
        fmt.Printf("Worker %d завершил задание %d\n", id, job)
    }
}
```

```
func main() {
    jobs := make(chan int)
    // Запускаем 3 воркера
    for i := 1; i <= 3; i++ {
        go worker(i, jobs)
    }

    // Отправляем 5 заданий
    for j := 1; j <= 5; j++ {
        jobs <- j
    }

    close(jobs) // важно закрыть канал
                // после отправки всех заданий

    time.Sleep(4 * time.Second)
}
```

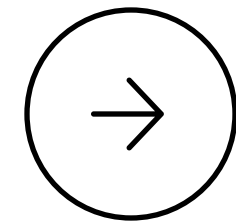
# Пример с пулом воркеров

- Канал `jobs` передаёт номера заданий
- Функция `worker` запускается в нескольких экземплярах
- Все воркеры читают задания из одного канала
- Кто первым прочитал задание, тот его и обрабатывает
- Такой подход позволяет распараллелить обработку без гонок данных
- Канал гарантирует, что каждое задание получит только один воркер

# Полезные правила работы с каналами

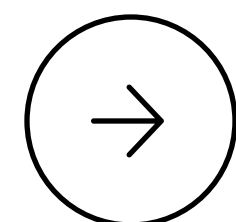
- Закрывайте канал, если он больше не нужен  
— особенно при использовании воркеров
- Никогда не записывайте в закрытый канал  
— это приводит к панике выполнения
- Не читайте из канала без горутины,  
если не уверены, что туда что-то запишут  
— иначе программа зависнет

# Итоги видео



## Сегодня мы потренировались:

- Передавать данные между горутинами с помощью каналов
- Координировать действия нескольких горутин через каналы (например, паттерн ring-pong)
- Использовать направленные каналы для ограничения доступа к отправке или получению данных
- Настраивать несколько воркеров для безопасного чтения из общего канала



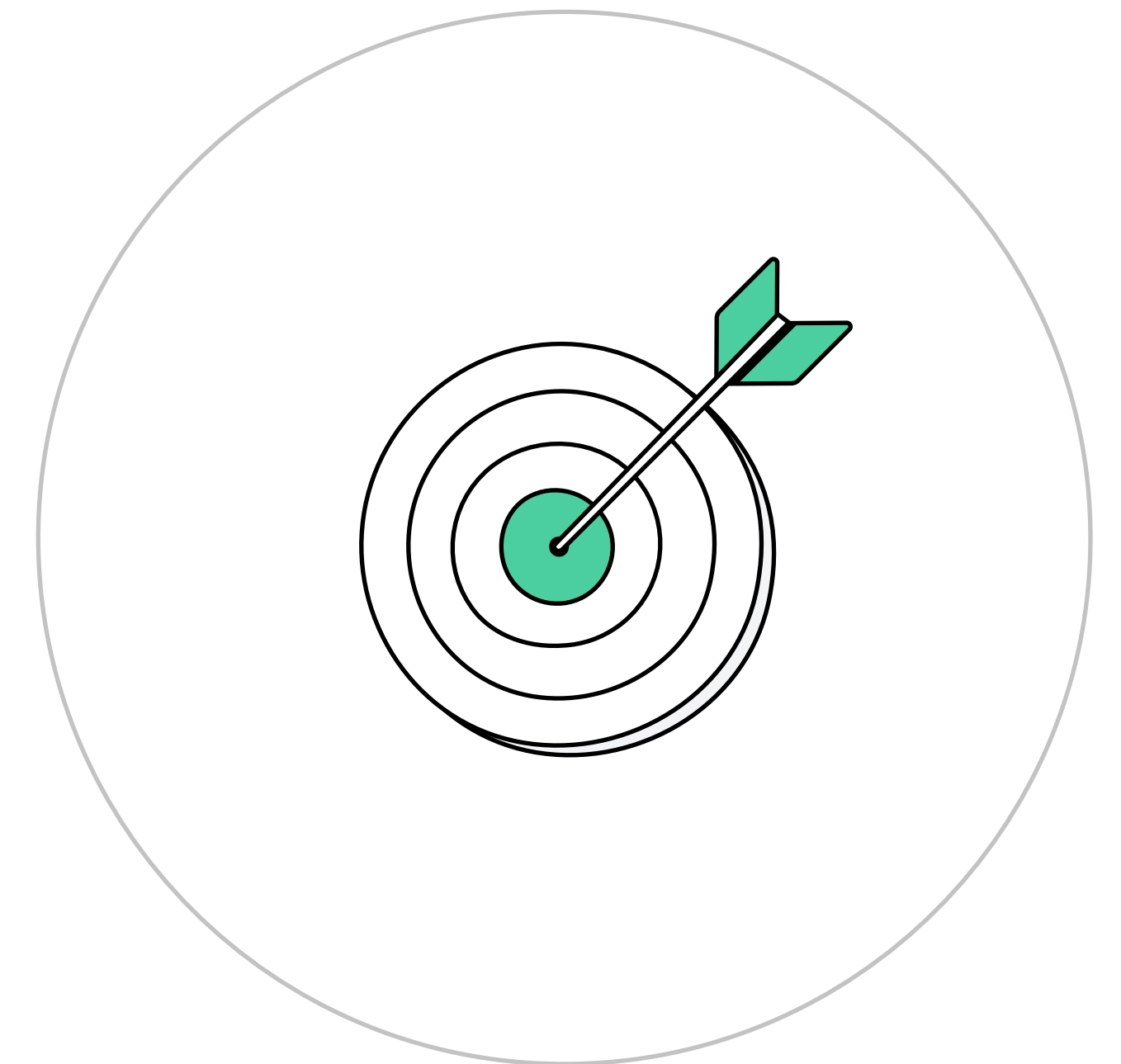
Это основа практики работы с каналами. Далее вас ждут примеры с более сложными структурами данных и управлением завершением работы горутин



# Работа с данными в канале: структуры и завершение

# Цели видео

- Сосредоточимся на работе с каналами как с потоками данных
- Разберёмся, как передавать через канал структуры
- Поймём, как правильно закрывать канал и что происходит при чтении после закрытия
- Это даст базу для написания устойчивых и читаемых параллельных программ



# Передача структур через канал

Что можно передавать в канал

- Простые значения: строки, числа
- Сложные значения: структуры с набором полей

Для реальных задач чаще используются  
структуры, описывающие объекты и сущности

# Пример 1: передаём Task через канал

Допустим, у нас есть структура задания

```
type Task struct {  
    ID    int  
    Name string  
}
```

# Пример 1: передаём Task через канал

```
package main

import (
    "fmt"
)

type Task struct {
    ID    int
    Name  string
}

func worker(tasks <-chan Task) {
    for task := range tasks {
        fmt.Printf("Обработка задания\n", task.ID, task.Name)
    }
}
```

```
}

func main() {
    taskChan := make(chan Task)

    go worker(taskChan)

    taskChan <- Task{ID: 1, Name:
        "Сделать отчёт"}

    taskChan <- Task{ID: 2, Name:
        "Отправить email"}

    taskChan <- Task{ID: 3, Name:
        "Обновить статус"}

    close(taskChan)
}
```

# Что происходит при передаче структур через канал

- Структура Task передаётся в канал как единый объект
- Горутина читает данные с помощью `for task := range tasks`
- После закрытия канала цикл завершает работу
- Такой подход даёт базу для потоковой обработки структурированных данных
- Логика легко масштабируется — можно запустить несколько горутин для параллельной обработки

# Пример с другой структурой: User

- Передаём в канал структуру User с полями ID и Email
- Каналы легко передают любые структуры
- Подходит для обработки сообщений, задач, запросов и других сущностей

```
type User struct {  
    ID      int  
    Email   string  
}
```

# Пример кода с передачей User

```
func processUsers(users <-chan User) {  
    for user := range users {  
        fmt.Printf("Пользователь %d, email: %s\n", user.ID, user.Email)  
    }  
}  
  
func main() {  
    users := make(chan User)  
    go processUsers(users)  
    users <- User{ID: 101, Email: "user1@example.com"}  
    users<-User{ID: 102, Email: "user2@example.com"}  
    close(users)  
}
```

# Заккрытие канала с помощью close(ch)

## Когда нужно закрыть канал

- Все данные уже отправлены
- Больше никто не будет писать в канал
- Получателю нужно знать, что данные больше не придут

## Важно

- Закрывать канал должен только отправитель
- После закрытия канал остаётся доступен для чтения, но не для записи

# Поведение при чтении из закрытого канала

Когда канал закрыт, попытка прочесть из него не вызывает панику, а возвращает нулевое значение типа и специальный флаг `ok = false`

```
func main() {  
    ch := make(chan int)  
    close(ch)  
    val, ok := <-ch  
    fmt.Println("val:", val, "ok:", ok)  
}
```

**Вывод:** `val: 0 ok: false`

# Конструкция `v, ok := <-ch`

Эта конструкция позволяет безопасно читать из канала, даже если он может быть закрыт

```
for {  
    val, ok := <-ch  
    if !ok {  
        break// канал закрыт  
    }  
    fmt.Println("Получено:", val)  
}
```

Часто вместо этого используется более короткая форма `range ch`, но `val, ok := <-ch` даёт полный контроль, если нужно обрабатывать нестандартные случаи

# Итерация по каналу с `range ch`

- Если вы закрыли канал и хотите прочитывать из него все значения, которые остались в очереди, используйте `range ch`
- Цикл `range` завершается сам, когда канал закрыт и все значения из него прочитаны. Это удобный и безопасный способ чтения из канала

# Пример: итерация по каналу с range ch

```
package main
import (
    "fmt"
)
func main() {
    ch := make(chan int)
    go func() {
        for i := 1; i <= 3; i++ {
            ch <- i
        }
    }()
```

```
    close(ch)
}()
for val := range ch {
    fmt.Println("Получено значение:",
        val)
}
}
```

# Типичные ошибки с каналами

Ошибка №1 — двойное закрытие канала

Пример кода:

```
ch := make(chan int)
close(ch)
close(ch) // ошибка!
```

В результате программа завершится с паникой: `close of closed channel`

Правило: закрывайте канал один раз, и только там, где вы управляете отправкой

# Типичные ошибки с каналами

Ошибка №2 — попытка записать значение в закрытый канал

Пример кода:

```
ch := make(chan int)
close(ch)
ch <- 1 // паника!
```

Программа завершится с паникой при выполнении: `send on closed channel`

Чтобы избежать: использовать флаг или `select` с каналом-сигналом остановки

# Сигнал завершения через канал

Сигнал завершения через канал

Пример кода:

```
done := make(chan struct{})  
go func() {  
    <- done  
    fmt.Println("Завершаюсь")  
}()  
time.Sleep(time.Second)  
close(done)
```

Здесь не передаём значения — только факт закрытия.  
Это типичный приём: закрытие канала как **сигнал**

# Итоги видео

- Работа с каналами включает передачу структур, например Task или User. Это помогает удобно организовать потоки данных в программе
- Для безопасного завершения работы с каналами мы используем функцию `close()`, чтобы явно сигнализировать о завершении отправки данных
- При чтении из канала мы применяем конструкцию `v, ok := <-ch`, чтобы контролировать ситуацию при закрытии канала
- Конструкция `range ch` позволяет безопасно прочитать все данные из канала до его закрытия — это упрощает обработку данных
- Типичные ошибки при работе с каналами — двойное закрытие, запись в закрытый канал или блокировка из-за отсутствия получателя — важно учитывать, чтобы избежать сбоев в работе программы

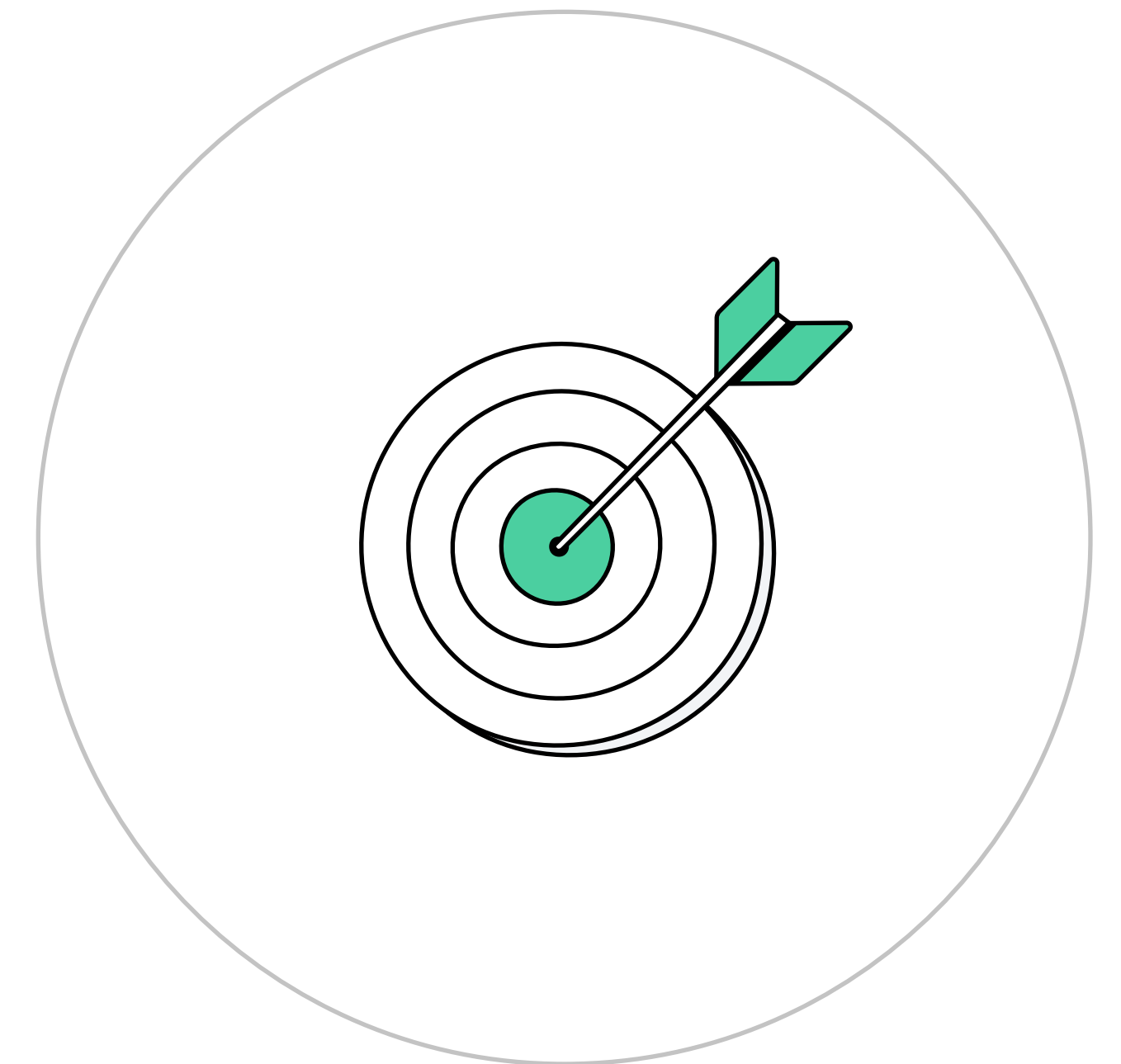


# Работа с каналами

Код, поведение горутин и типичные ошибки

# Цели видео

- Разберём примеры кода с каналами и взаимодействием горутин
- Проанализируем поведение горутин и способы координации их работы
- Разберём типичные ошибки при работе с каналами и способы их избежать



# Ключевые конструкции каналов в Go

## На что обращать внимание в коде с каналами:

- `make(chan T)` — создание канала
- `ch <- val` — отправка значения в канал
- `<-ch` — получение значения из канала
- `range ch` — чтение из канала до закрытия
- `close(ch)` — сигнал о завершении передачи данных
- `val, ok := <-ch` — безопасное чтение с проверкой закрытия
- `chan<-` и `<-chan` — направленные каналы, только на запись или только на чтение

Эти конструкции используются во всех Go-проектах

# Пример ping-pong с каналами

Простой паттерн взаимодействия горутин с использованием каналов

Пример кода:

```
package main
import (
    "fmt"
    "time"
)
func main() {
    ping := make(chan string)
    pong := make(chan string)
    go func() {
        for {
            msg := <-ping
            fmt.Println("Ping получил:", msg)
            time.Sleep(500 * time.Millisecond)
            pong <- "от ping"
        }
    }()
```

```
    }
}()
go func() {
    for {
        msg := <-pong
        fmt.Println("Pong получил:", msg)
        time.Sleep(500 * time.Millisecond)
        ping <- "от pong"
    }
}()
ping <- "старт"
time.Sleep(3 * time.Second)
fmt.Println("Завершаем")
```

# Что важно в ping-pong-примере

## Особенности работы кода:

- Цикл запускается: ping → pong → ping → ...
- Горутины никто не останавливает  
— это допустимо в демонстрации
- Для продакшена лучше использовать сигнальные каналы
- Каналы небуферизированные, передача данных синхронная

# Передача структур через канал

Пример с передачей структуры Task в канал и обработкой в воркере

Пример кода:

```
func worker(id int, jobs <-chan int, results chan
<-string) {
    for job := range jobs {
        result := fmt.Sprintf("Worker %d обработал
задание %d", id, job)
        time.Sleep(200 * time.Millisecond)
        results <- result
    }
}
func main() {
    jobs := make(chan int)
    results := make(chan string)
    for w := 1; w <= 3; w++ {
        go worker(w, jobs, results)
    }
}
```

```
go func() {
    for j := 1; j <= 6; j++ {
        jobs <- j
    }
    close(jobs)
}()
for i := 0; i < 6; i++ {
    fmt.Println(<-results)
}
}
```

# Что происходит в примере с Task

## Ключевые моменты работы кода:

- В канал `tasks` передаются три структуры `Task`
- Воркер читает задачи с помощью `range` и обрабатывает их
- После закрытия канала `tasks` цикл завершает работу автоматически
- Канал `done` сигнализирует о завершении воркера

# Очередь задач и несколько воркеров

Пример с обработкой задач несколькими воркерами из общего канала

Пример кода:

```
type Task struct {
    ID    int
    Name  string
}
func worker(tasks <-chan Task, done chan<-int) {
    for task := range tasks {
        fmt.Printf("Выполняю задачу #%d: %s\n",
            task.ID, task.Name)
        time.Sleep(300 * time.Millisecond)
    }
    done <- 1
}
```

```
func main() {
    tasks := make(chan Task)
    done := make(chan int)
    go worker(tasks, done)
    tasks <- Task{ID: 1, Name: "Проверить логи"}
    tasks <- Task{ID: 2, Name: "Обновить систему"}
    tasks <- Task{ID: 3, Name: "Перезапустить сервис"}
    close(tasks)
    <-done
    fmt.Println("Все задачи обработаны")
}
```

# Что происходит в примере с несколькими воркерами

## Особенности работы кода:

- в канал `jobs` отправляется 6 заданий
- результаты собираются из канала `results` в количестве 6 штук
- три воркера обрабатывают задания параллельно
- порядок получения результатов может быть произвольным — это нормально

# Ошибка: чтение из канала без записи

Типичная ошибка — чтение из канала без записи

Пример кода:

```
ch := make(chan int)
val := <-ch // зависнет навсегда
```

Что происходит:

- программа зависает в ожидании данных
- никто не пишет в канал, горутина блокируется навсегда

# Ошибка: запись в канал, когда никто не читает

Что происходит:

- канал создан, но в него никто не пишет
- программа зависает навсегда

```
ch := make(chan int)
```

```
ch <- 1 // блокируется, если никто не читает
```

# Ошибка: попытка читать из закрытого канала, не проверяя ok

## Что происходит:

- чтение из закрытого канала даёт нулевое значение типа
- программа продолжает работать с невалидными данными
- пример кода:

```
val := <-ch  
// а канал уже закрыт
```

## Почему это опасно:

- ошибка не вызывает панику
- результат чтения может привести к логическим ошибкам

# Ошибка: попытка читать из закрытого канала, не проверяя ok

Как исправить:

используйте безопасное чтение  
с проверкой закрытия канала

```
val, ok := <-ch
if !ok {
    // канал закрыт
}
```

# Самостоятельная тренировка по каналам

## Задача для закрепления материала:

Напишите программу, где:

- создаются две горутины-воркера
- через канал `chan Task` передаётся 5 заданий
- каждый воркер печатает ID задачи и своё имя
- `main()` ждёт завершения всех задач, прежде чем выйти

## Подсказки:

- используйте `range` для чтения из канала
- закройте канал после отправки всех заданий
- для синхронизации завершения используйте канал `done`

# Итоги видео

- Разобрали распространённые паттерны работы с каналами: ring-pong, передача структур и несколько воркеров
- Проанализировали ключевые конструкции для работы с каналами: make, отправка и получение данных, range, close и v, ok := <-ch
- Обсудили типичные ошибки при работе с каналами и способы их избежать
- Получили финальное задание для самостоятельной тренировки — с многими из показанных приёмов



# Итоги



# Итоги занятия

- Мы узнали, что каналы позволяют безопасно передавать данные между горутинами — это даёт возможность организовать взаимодействие между потоками без гонок данных
- Мы разобрались, что `range` и `ok` помогают корректно читать данные из канала — это даёт инструмент для чистого завершения работы с каналом
- Мы поняли, что закрытие канала — это сигнал о завершении передачи данных — это помогает строить устойчивую логику завершения программы
- Мы узнали о типичных ошибках с каналами — это даёт возможность избегать зависаний и падений программы
- Мы разобрали паттерны `ping-pong` и `worker pool` — это даёт примеры построения структурированной многопоточной логики в Go



# Отправка и получение данных через канал

Матвей Ефимов

Разработчик учебных программ и эксперт в Go, Python, SQL

